UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN

ECE408 - CS483 - CSE408 Spring 2026
Applied Parallel Programming

Lecture 1: Introduction

Volodymyr Kindratenko

1/20/2026

Before We Get Started

- Welcome to the course!
- The course is taught in-person
  - Lectures are in-class; they are also recorded and posted on Mediaspace
  - Labs (aka MPs) & Projects are on your own activities
  - Midterm exams are in-person
- Lecture slides will be posted on-line prior to the lecture on the course's Canvas page
  - https://canvas.illinois.edu/courses/66336
- The slides used in this class are based on the original version created by David Kirk (NVIDIA) and Wen-mei Hwu (now at NVIDIA), 2007-2018, with contributions from Steven Lumetta (UIUC) and Sanjay Patel (now at NVIDIA)

Course Staff

Instructor: Prof. Volodymyr Kindratenko <kindrtnk@illinois.edu>

TAs: Xiyue Zhu (xiyuez2@illinois.edu), Hrishi Shah (hrishis2@illinois.edu),
Zhixiang Liang (zliang18@illinois.edu), Chiming Ni (chiming2@illinois.edu)
Nicholas Satchanov (nsatch3@illinois.edu)

UCAs: Charles Tianwei Pei (ctpei2@illinois.edu), Siyuan Gong (siyuang3@illinois.edu),
Hamzah Jamal (hamzahj2@illinois.edu), Evan Lin (elin26@illinois.edu),
Yifei Li (yifei28@illinois.edu)

About Me

- Ph.D. from University of Antwerp, Belgium, 1997
- At NCSA since 1997
  - Director of the Center for AI Innovation (CAII)
  - Research: Computing Systems, HPC, Computational Accelerators (FPGAs, GPUs), ML/AI systems & applications

AC – first GPU HPC cluster built in 2008 (used to teach this course too)
- 32 S1070 GPUs

HAL – first AI-oriented cluster built in 2018
- 64 V100 GPUs

DeltaAI – lates GPU HPC cluster built in 2024
- Grace-Hopper nodes based on the GH200 superchip

Course Goals

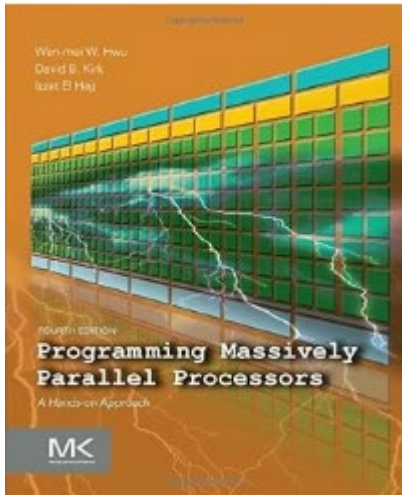
- Learn to program massively parallel processors and achieve
  - High performance
  - Functionality and maintainability
  - Scalability across future generations
- Technical subjects
  - Parallel programming basics
  - Principles and patterns of parallel algorithms
  - Programming API, tools and techniques
  - Processor architecture features and constraints
  - Killer apps (aka AI)

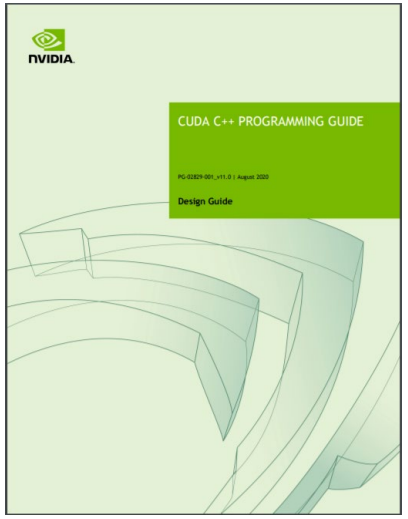
Web Resources

- **Canvas** (https://canvas.illinois.edu/courses/66336)
  - Syllabus, lecture slides, links to recordings, course files, etc.
  - Project reports
  - Grades
- **Illinois Media Space** (https://mediaspace.illinois.edu/channel/channelid/397193873)
  - Recorded lectures
- **GitHub** (https://github.com/illinois-cs-coursework/sp26_ece408_.release)
  - Lab and project assignments
- **PrairieLearn** (https://us.prairielearn.com)
  - Lab quizzes
- **Campuswire** (https://campuswire.com/c/G794D32E4/feed)
  - Channel for electronic announcements
  - Forum for Q&A – staff and your classmates can post questions and answers
- **Course Chatbot** (https://chat.illinois.edu/ECE408SP26/chat)
  - Ask the chatbot first before posting a question on campuswire or going to the office hour

Course Text/Notes

- Textbook
  - Wen-mei Hwu, David Kirk and Izzat El Hajj, “Programming Massively Parallel Processors: A Hands-on Approach,” Morgan Kaufman Publisher, 4th edition, 2022, ISBN 978-0-323-91231-0.
  - The book can be downloaded from ScienceDirect
- NVIDIA documentation (v.12.8)
  - NVIDIA, CUDA C Programming Guide (CUDA PG)
  - NVIDIA, CUDA C++ Best Practices Guide (CUDA BPG)
  - Available at NVIDIA Documentation website





Typical Course Schedule

- Week 1:
  - Tuesday: Lecture 1: Introduction
  - Thursday: Lecture 2: CUDA Intro
  - GitHub and computer access setup, Lab 0
- Week 2:
  - Tuesday: Lecture 3: Data Parallelism Model
  - Thursday: Lecture 4: CUDA Memory Model
  - Lab 1 due: VectorAdd example
- Week 3:
  - Tuesday: Lecture 5: CUDA Memory Model
  - Thursday: Lecture 6: Performance Considerations
  - Lab 2 due: Simple Matrix-Matrix multiplication

all *Labs (MPs)* and *Project Milestones* (PMs) are due on Fridays at 8:00pm US Central Time

Late Submission Policy

- All assignments are due by Friday at 8pm. However, recognizing potential issues such as Delta system maintenance, interviews, or illness, we provide a **three-day grace period**.
  - You can submit an assignment by Monday at 8pm and still receive full credit without penalty. You do not need to notify us if you intend to use this default 3-day grace period.
  - However, to extend your grace period beyond 3 days, you must notify us by Friday 8pm via email to the TA(s) responsible for the assignment you are asking about. Requests after this time will not be considered. Extensions to the grace period are given out for extenuating circumstances at our discretion.
  - Please also note that the code in your GitHub repository at 8pm on Monday will be considered the final version, even if you had a correct version earlier. We will retrieve and grade the code at that time.

Grading

- Exams: 50%
  - Midterm 1: 25% -- ~ first 12 lectures
  - Midterm 2: 25% -- ~ the remaining lectures
- Labs (Machine Problems): 25%
  - Passing Datasets 90%
  - Correct answers to questions
  - Lowest graded lab will be dropped
  - Labs 0 and 1 are not counted in the final grade
- Project: 25%
  - Demo/Functionality/Coding Style: ~50%
  - Performance with full functionality: ~50%
  - Detailed rubric will be posted

Computer System for Assignments

- Delta supercomputer operated by NCSA will be used for working on labs (MPs) and projects
  - You will need to obtain an account on this system first
  - GitHub repo includes all the details about setting up the lab environment
- All Labs (MPs) and Project Milestones (PMs) are due on Fridays at 8:00pm US Central Time
  - **Lab 0 is due this Friday at 8pm**
- Lab workflow:
  - #1: Get base code from GitHub → write your code & compile & run on Delta → submit final code for grading to GitHub
  - #2: Answer questions on PrairieLearn **AFTER** #1 is completed

NCSA Delta Supercomputer

- Delta is a dedicated, ACCESS-allocated resource designed by HPE and NCSA, delivering a highly capable GPU-focused compute environment for GPU and CPU workloads.

4-Way **A40 GPU Compute Node** Specs	
Specification	**Value**
Number of nodes	100
GPU	NVIDIA A40
GPUs per node	4
GPU Memory (GB)	48 DDR6
CPU	AMD Milan
CPU sockets per node	1
Cores per socket	64
Cores per node	64
Clock rate (GHz)	~ 2.45
RAM (GB)	256

Getting Started with Delta

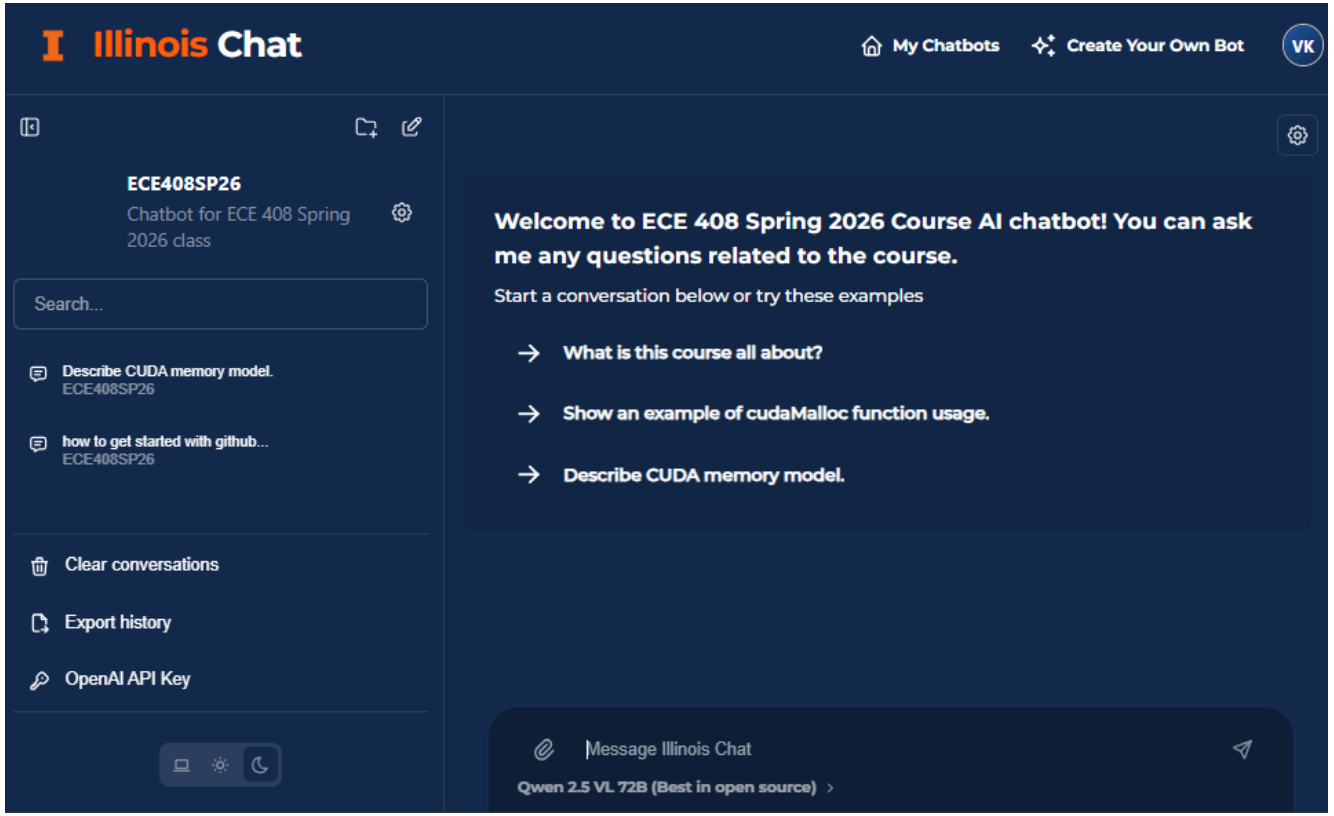
- Go to https://identity.access-ci.org/new-user and register yourself as a new user.
  - If you already have an ACCESS account (some of you might already have it due to a research project, etc.) do not register as new user, just use the same username.
  - When registering for new account, use "University of Illinois Urbana-Champaign" as your Identity Provider. This way, you can just use your UofI credentials.
- Once your account is created, note your ACCESS user ID and submit this user ID to the course staff via this form: https://forms.illinois.edu/sec/164314864.
- Wait for an email from the Delta team confirming completion of your account registration. This may take anywhere between 24 to 48 hours.
  - Start early, do not delay with getting access do Delta!

Getting Started with Delta (cont'd)

- After you receive an email from the Delta team about your account, you should be able to login into Delta's head node login.delta.ncsa.Illinois.edu:
  - `ssh yourusername@login.delta.ncsa.illinois.edu`
  - DUO is needed for 2FA
- Read Delta's documentation at https://docs.ncsa.illinois.edu/systems/delta/en/latest/
- On Delta, clone course's GitHub repository and follow the instructions for editing, compiling, running and submitting your work. Important:
  - Compile on the head node
  - Submit for execution to one of the compute nodes

Course AI Chatbot

- https://chat.illinois.edu/ECE408SP26/chat
- Developed by the CAII team at NCSA
- Uses GPT models via OpenAI API or locally run open-source models
- Context Stuffing and Mega-Prompting (with course materials)
- Please read the disclaimer posted on the course's website before using the chatbot!



Academic Honesty

- You are allowed and encouraged to discuss assignments with other students in the class. Getting verbal advice/help from people who've already taken the course is also fine.
- Any reference to assignments from previous terms or web postings is unacceptable.
- Any copying of non-trivial code is unacceptable
  - Non-trivial = more than a line or so
  - Copying includes reading someone else's code, including code produced by a chatbot, and then going off to write your own.
  - Those who have allowed copying will also be penalized

Academic Honesty (cont'd)

- Giving/receiving help on an exam is unacceptable.
- Deliberately sidestepping the lab requirements is unacceptable.
- Penalties for academic dishonesty:
  - Zero on the assignment/exam for the first occasion
  - Reported to the College via FAIR system
  - Automatic failure of the course for repeat offenses

Permitted Use of Class AI Chatbot

- Asking about CUDA API, e.g.:
  - “What does cudaMemcopy do?” or “How do I call a CUDA kernel?”
- Asking about CUDA programming techniques, e.g.:
  - “How do I declare an array in shared memory?”, or “How do I pass shared memory array size at the run-tine?”
- Asking about parallel programming concepts, performance optimization techniques, etc., e.g.:
  - “Explain how tiling helps with global memory bandwidth.”
- Asking to help with compiler and run-time errors
- Asking questions related to midterm exam preparation
- Asking about general course details

NOT Permitted Use of Class AI Chatbot

- Asking to generate code for an assignment, e.g.:
  - "Give me the CUDA kernel code for tiled matrix multiplication from Lab 3."
- Copy-paste lab quiz questions into the chatbot.
  - In many cases, the answer will not be correct anyway because the chatbot is not aware of all the assumptions.
- Asking chatbot to fix bugs in your code by copy-pasting your entire code into the chatbot.

- Do not file regrade requests afterwards if you get a poor grade because the chatbot gave you an incorrect answer!

Post-Dennard technology pivot – parallelism and heterogeneity

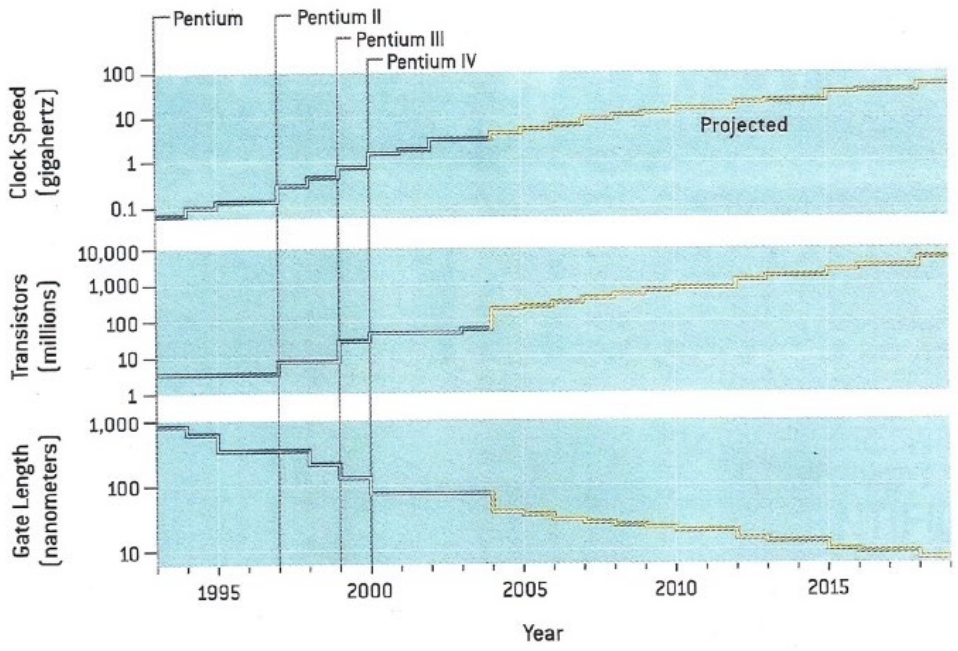
The Moore's Law Era

- Moore's "Law" (Imperative) drove features sizes down, doubling the number of transistors/unit area every 18-24 months
- Dennard Scaling (based on the physics) drove clock speeds up
- Moore's Law equated in popular press with performance
  - This was really a combination of Moore's Law + Dennard Scaling
- For about 30 years, general purpose computers doubled in performance every 18 months
  - Very hard to keep up – any "special purpose" computer might start with a performance advantage – but lose it over the next few years
  - Industry littered with computer companies that tried to get ahead of Moore+Dennard
  - Led to "Attack of the Killer Micros"

Slides curtesy of Prof. William Gropp, NCSA Director

What The Moore's Law Era Gave Us

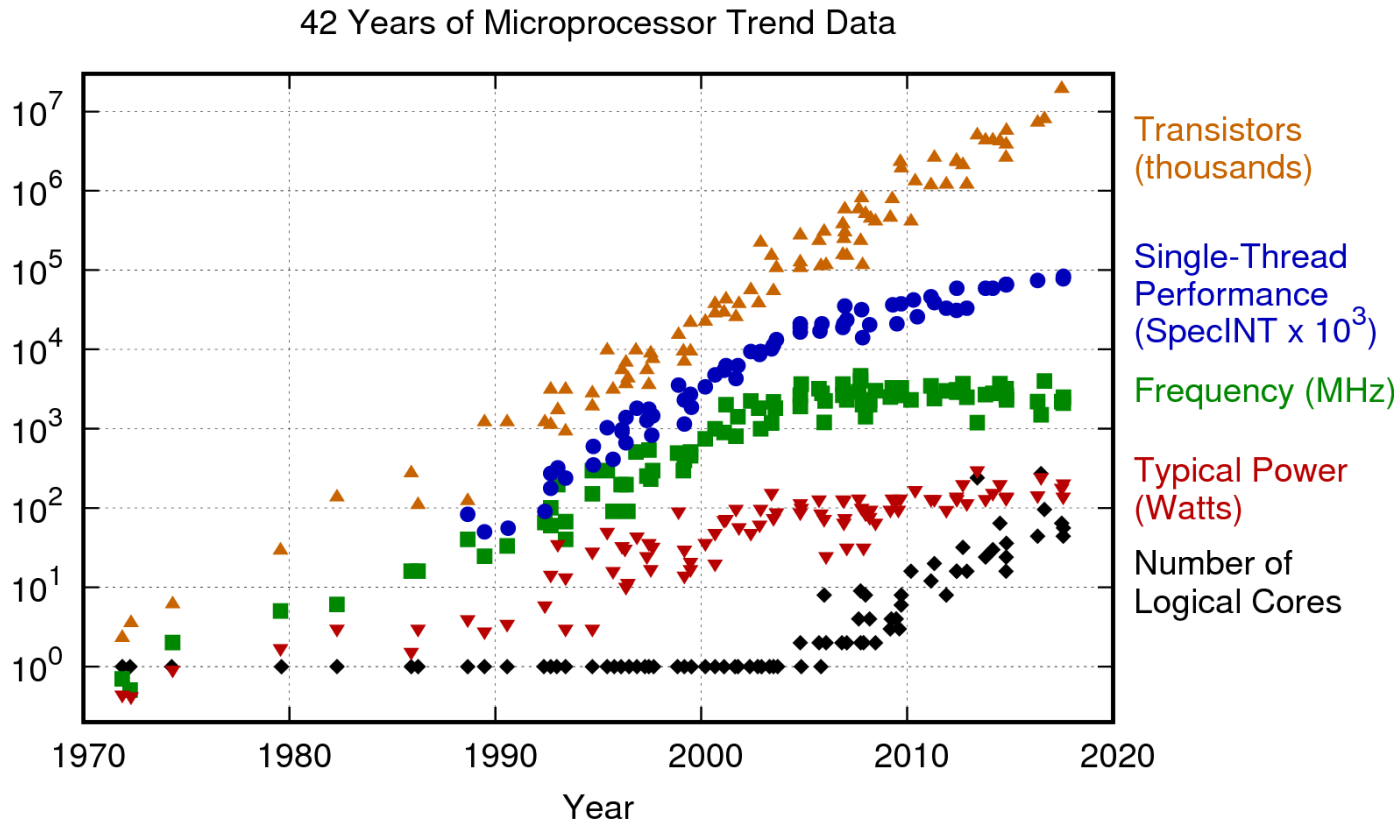
- Exponential increase in clock speed
  - Single core performance doubled every 18 months
  - Code not fast enough? Just wait 2 years!
- Software …
  - Same software ran faster and faster
    - Unless memory-latency bound, as in some sparse, unstructured or graph codes
  - Increase in density meant memory went from a scarce, expensive resource to one that, *for most applications,* was effectively infinite
    - That changed many aspects of programming and algorithms
- Stability in software, combined with "automatic" performance increases, meant application codes could evolve over time. Portability of software expected.



Slides curtesy of Prof. William Gropp, NCSA Director

End of Easy Performance Gains

- Dennard (Frequency) Scaling ended around 2005-2006
  - Previously neglected effect, leakage current, became important
  - Clock speeds stagnated at 2.5-3.5 GHz
  - Almost 20 years later, clock speeds are nearly the same
- But the expectation of doubling of performance was strong
  - And Moore's "Law", though slowing down, is still providing higher component density
- Maintaining performance expectation required doing something different
  - Instruction parallelism (vectors)
  - Multiple cores per "processor"
  - Specialization, especially to vectors/streams/GPUs



Original data up to the year 2010 collected and plotted by M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, and C. Batten
New plot and data collected for 2010-2017 by K. Rupp

Slides curtesy of Prof. William Gropp, NCSA Director

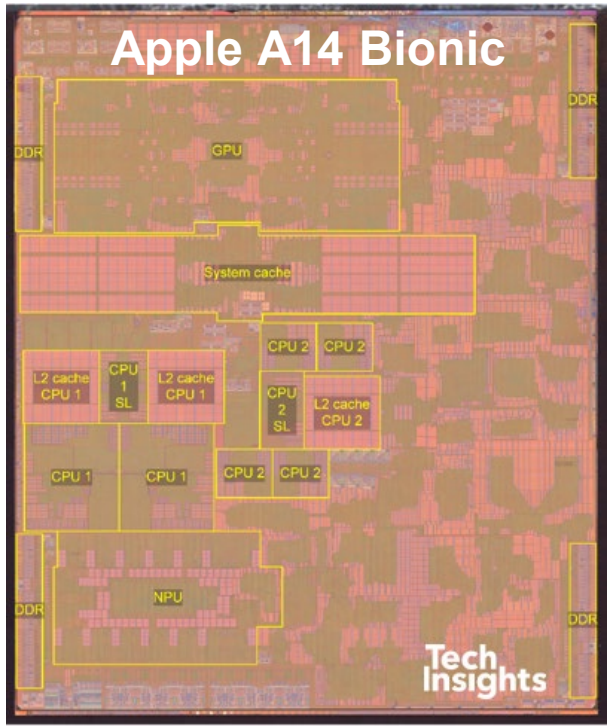
Post-Dennard Pivoting - Processor Specialization

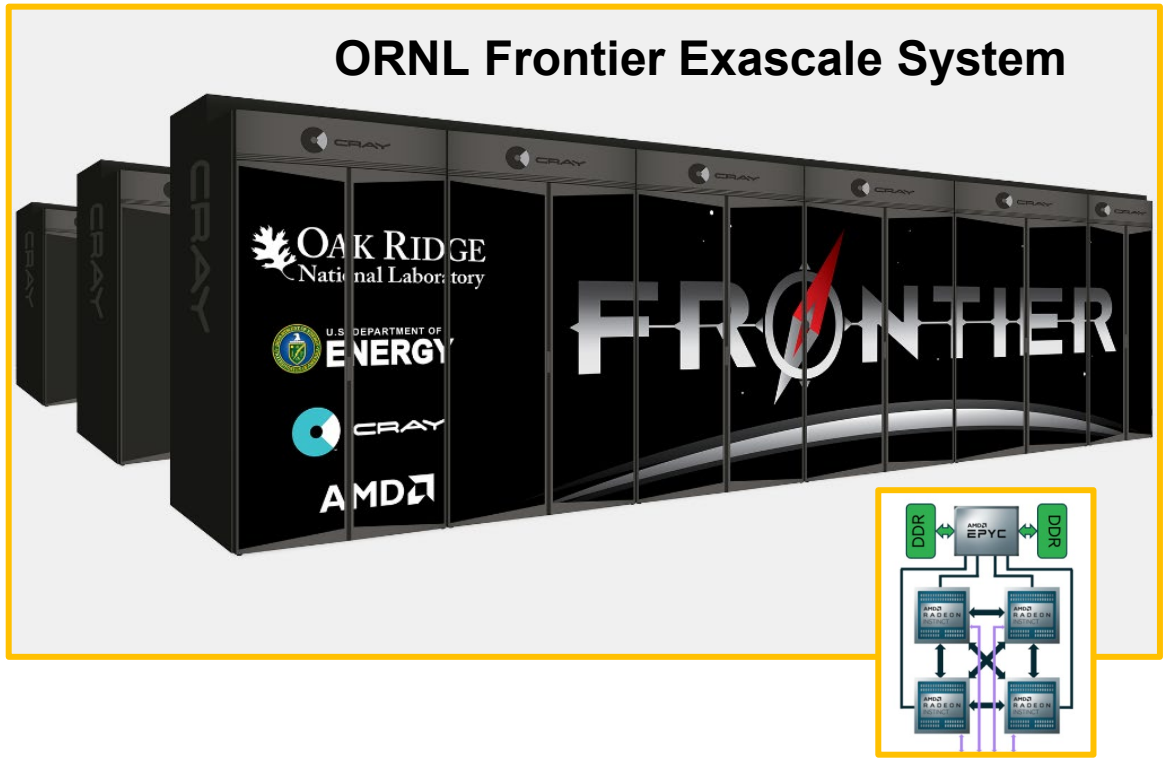
- Conventional CPUs are general purpose
  - Run anything, ok to great performance on most codes
- But with end of Dennard scaling, not all applications continued to see more performance
  - Parallelism required to maintain exponential increase in performance
  - Some attempts at lightweight processing cores - simpler cores and more of them
    - IBM Blue Gene and Intel Xeon Phi in HPC
- Another direction: do not insist on running most codes (well)
  - Specialize to a subset of operations
  - GPUs were an early example, building on previous work in graphics and SIMD parallelism
  - Your phone is another example, with processors optimized for power and important (to the mobile phone user) operations
- HPC is a fertile ground for *some* specialization (recall Vector supercomputers)

Slides curtesy of Prof. William Gropp, NCSA Director

The Trend is Across the Board

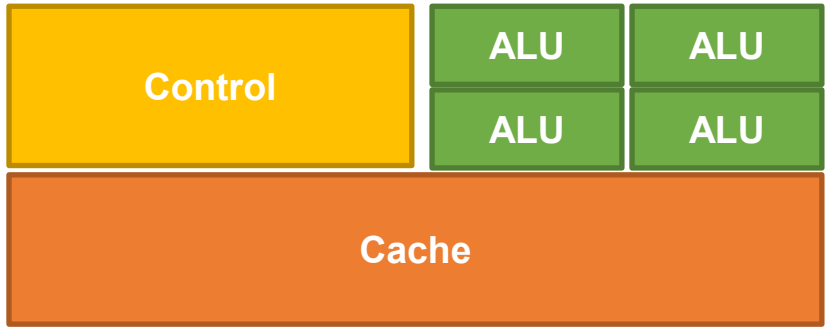
- From the smallest to the largest, computing today involves accelerators architectures





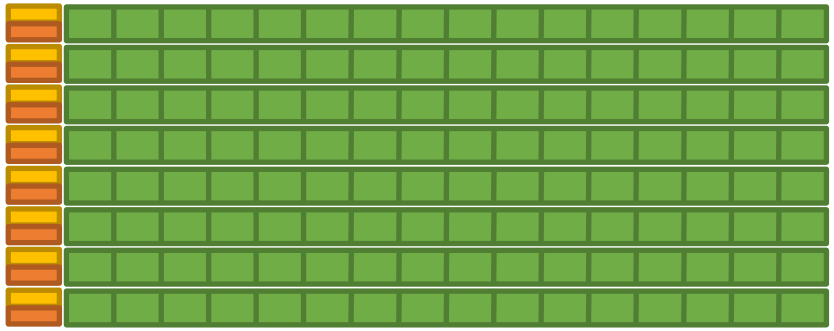
Processor Design Approaches

CPU: Latency-Oriented Design



- A few powerful **ALUs**
  - Reduced operation latency
- Large **caches**
  - Convert long latency memory accesses to short latency cache accesses
- Sophisticated **control**
  - Branch prediction to reduce control hazards
  - Data forwarding to reduce data hazards
- Modest multithreading to hide short latency
- High clock frequency

GPU: Throughput-Oriented Design

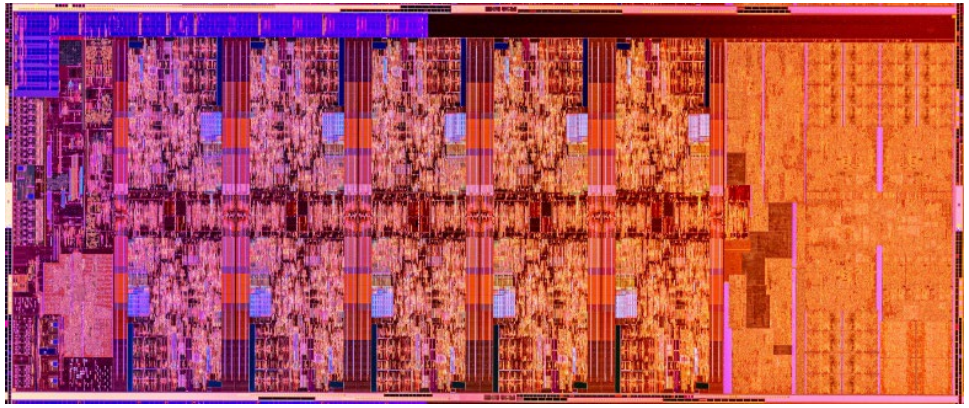


- Many small ALUs
  - Long latency, high throughput
  - Heavily pipelined for further throughput
- Small **caches**
  - More area dedicated to computation
- Simple **control**
  - More area dedicated to computation
- Massive number of threads to hide the very high latency!
- Moderate clock frequency

Winning Strategies Use Both CPU & GPU

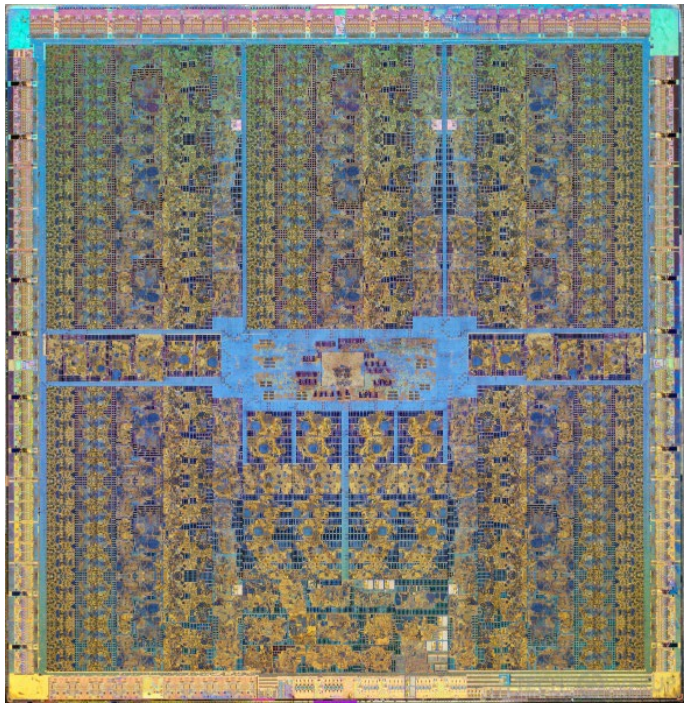
CPU

- 10th Gen Intel Core processor
  - 10 cores silicon
  - 14 nm process



GPU

- NVIDIA GK110
  - 2,880 CUDA cores
  - 28 nm process



CPU vs GPU

CPU

- CPUs for sequential parts where latency hurts
  - CPUs can be 10+X faster than GPUs for sequential code

GPU

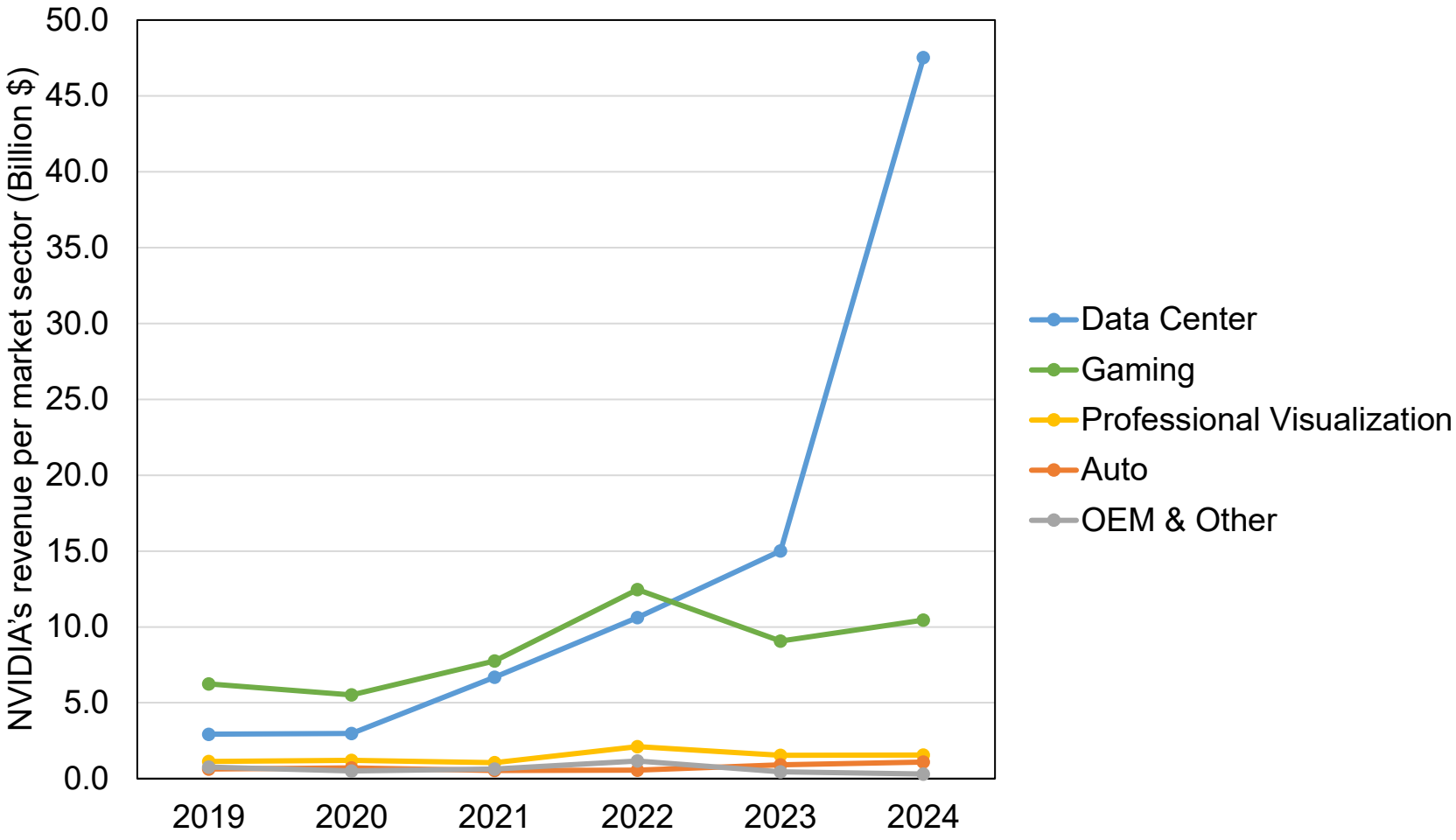
- GPUs for parallel parts where throughput wins
  - GPUs can be 10+X faster than CPUs for parallel code

GPU Origins

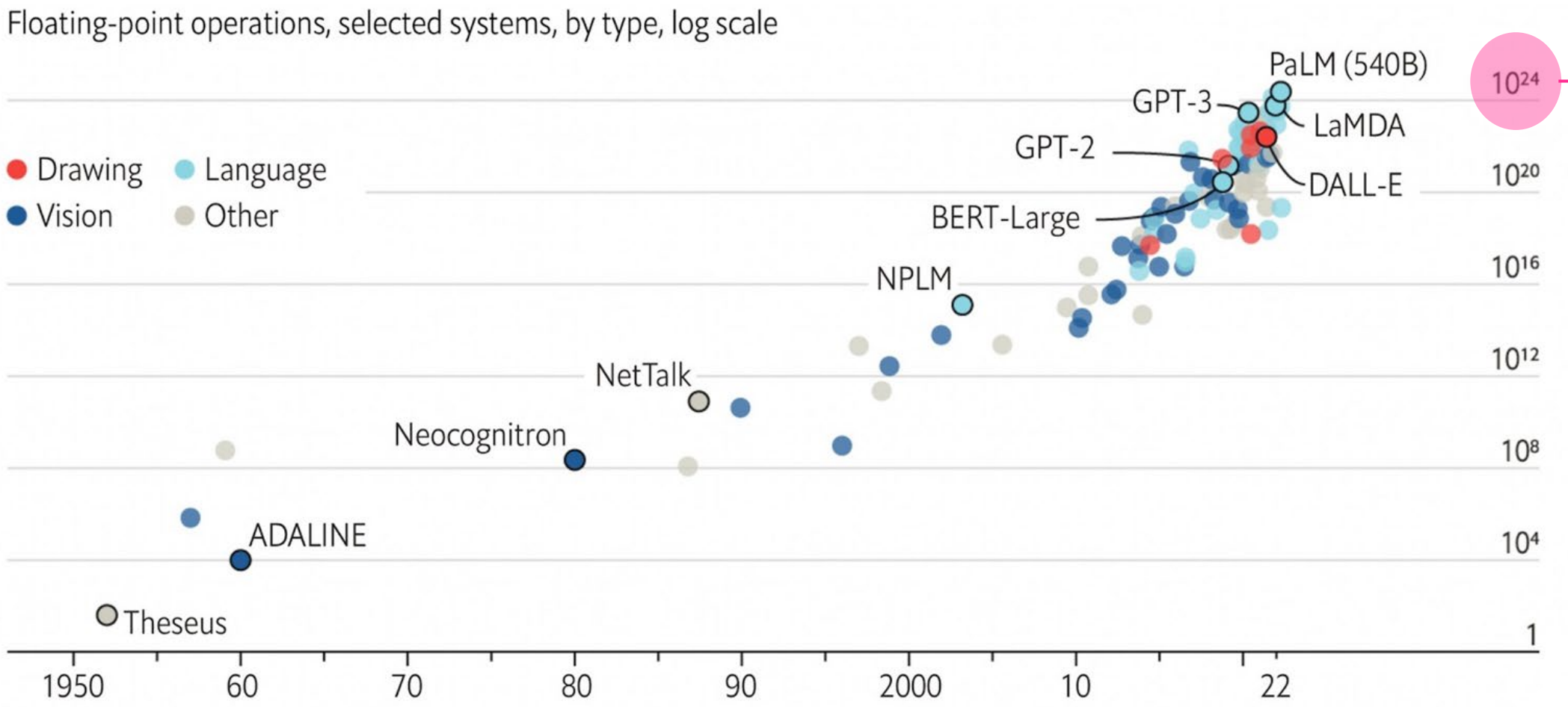
- GPUs were originally built for graphics
  - Hence the name, Graphics Processing Unit
  - Graphics workloads process a massive number of pixels in parallel
- Before 2007, the only way to program GPUs was using graphics APIs
  - e.g., OpenGL, Direct3D
  - Using GPUs for other than graphics meant reformulating a computation as a function that paints a pixel
- In 2007, NVIDIA released CUDA
  - A programming interface for using GPUs in a general-purpose way
  - Requires extensions to the GPU architecture

Growth of General-Purpose GPU Computing

- Major growth in data center revenue indicates growth of general-purpose GPU computing, particularly fueled by artificial intelligence



Computing Needs for AI



10^{24}

Sources: "Compute trends across three eras of machine learning", by J. Sevilla et al., arXiv, 2022; Our World in Data

Prevalence of GPUs in Supercomputers

- June 2025 Top-500 List
  - #1: El Capitan - HPE Cray EX255a, AMD 4th Gen EPYC 24C 1.8GHz, **AMD Instinct MI300A**, Slingshot-11, TOSS, HPE
  - #2: Frontier - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, **AMD Instinct MI250X**, Slingshot-11, HPE Cray OS, HPE
  - #3: Aurora - HPE Cray EX - Intel Exascale Compute Blade, Xeon CPU Max 9470 52C 2.4GHz, **Intel Data Center GPU Max**, Slingshot-11, Intel
  - #4: JUPITER Booster - BullSequana XH3000, GH Superchip 72C 3GHz, **NVIDIA GH200 Superchip**, Quad-Rail NVIDIA InfiniBand NDR200, RedHat Enterprise Linux, EVIDEN
  - #5: Eagle - Microsoft NDv5, Xeon Platinum 8480C 48C 2GHz, **NVIDIA H100**, NVIDIA Infiniband NDR, Microsoft

https://www.top500.org

Why GPUs?

- Why repurpose a graphics processing architecture instead of designing a throughput-oriented architecture from scratch?

- Chips are expensive to build and require a large volume of sales to amortize the cost
  - This makes the chip market very difficult to penetrate

- When parallel computing became mainstream, GPUs already had (and still have) a large installed base from the gaming sector
  - Gave them a head start compared to other potential massively parallel accelerators

Parallel Programming Frameworks

- In this class, we will focus on CUDA
- Other GPU parallel programming frameworks exist:
  - OpenCL
  - OpenACC
  - …
- Popular CPU programming frameworks
  - MPI (distributed memory system)
  - OpenMP (shared memory system)
  - Both can be used in conjunction with CUDA

Parallel Programming Workflow

- Identify compute intensive parts of an application
- Adopt/create scalable algorithms
- Optimize data arrangements to maximize locality
- Performance Tuning
- Pay attention to code portability, scalability, and maintainability

Parallel Computing Challenges

- Massive Parallelism demands Regularity

- Load Balance
  - The total amount of time to complete a parallel job is limited by the thread that takes the longest to finish

good bad!

Parallel Computing Challenges (cont'd)

- Global Memory Bandwidth
  - Ideal vs. Reality

Ideal

Reality

- Conflicting Data Accesses Cause Serialization and Delays
  - Massively parallel execution cannot afford serialization
  - Contentions in accessing critical data causes serialization

Parallel Computing Pitfall

- Consider an application where:
  - The sequential execution time is 100s
  - The fraction of execution that is parallelizable is 90%
  - The speedup achieved on the parallelizable part is 1000×
- What is the overall speedup of the application?

$$t_{parallel} = (1 - \mathbf{0.9}) * \mathbf{100}s + \frac{\mathbf{0.9} * \mathbf{100}s}{\mathbf{1000}} = 10.09s$$

$$speedup = \frac{t_{sequential}}{t_{parallel}} = \frac{\mathbf{100}s}{10.09s} = 9.91 \times$$

Amdahl's Law

- In general, if an application has:
  - t sequential execution time
  - p fraction of execution that is parallelizable
  - s speedup achieved on the parallelizable part
- What is the overall speedup of the application?

$$t_{parallel} = (1 - p) * t + \frac{p * t}{s} = \left(1 - p + \frac{p}{s}\right) * t$$

$$speedup = \frac{t_{sequential}}{t_{parallel}} = \frac{t}{\left(1 - p + \frac{p}{s}\right) * t} = \frac{1}{1 - p + \frac{p}{s}} \xrightarrow{s \to \infty} \frac{1}{1 - p}$$

Amdahl's Law Implications

- The maximum speedup of a parallel program is limited by the fraction of execution that is parallelizable

$$speedup < \frac{1}{1 - p}$$

  - e.g., if p is 90%, speedup < 10×

- Fortunately, for many real applications, p > 99% especially for large datasets, and speedups >100× are attainable

What is the stake?

- Scalable and portable software lasts through many hardware generations

- Scalable algorithms and libraries can be the best legacy we can leave behind from this era

Any More Questions?

Things to Read:

Textbook chapter 1

CUDA PG: Introduction

Things to Do:

Get Delta account

Setup GitHub

Submit Lab 0